

เอกสารประกอบ — ภาควิทยาศาสตร์และเทคโนโลยี

Software Development Life Cycle (SDLC) & System Design

ระบบอ้างอิง: Sport Store — เว็บไซต์คอมเมิร์ซขายอุปกรณ์กีฬา (Next.js Frontend / Fastify + Supabase Backend / ชำระเงินผ่าน PromptPay QR)

w4_T003_67159957_paloch

คำถาม 1 — การประยุกต์ใช้เครื่องมือในกระบวนการ SDLC (2 คะแนน)

1) Planning — การวางแผน

เครื่องมือ	เหตุผลที่เลือกใช้
GitHub Projects / Trello	แบ่งงาน (Frontend/Backend), ทำ Kanban board, ติดตามสถานะแต่ละฟีเจอร์
Google Docs / Sheets	เก็บ Requirement, Timeline, แบ่งความรับผิดชอบตามตารางผู้จัดทำ
Discord / LINE	สื่อสารในทีม นัดประชุม แจ้งความคืบหน้า

ตัวอย่างในโครงการ: ใช้ GitHub Projects แดกงานเป็นหมวด Frontend (หน้าร้าน/ตะกร้า/บัญชี) และ Backend (API สินค้า/คำสั่งซื้อ/ชำระเงิน) มอบหมายให้สมาชิกแต่ละคนตาม role

2) Analysis & Design — การวิเคราะห์และออกแบบ

เครื่องมือ	เหตุผลที่เลือกใช้
draw.io (diagrams.net) / StarUML	วาด UML: Use Case, Class, Sequence — ฟรี ใช้งานง่าย export รูปได้
Figma	ออกแบบ Wireframe และ Prototype แบบกดใช้งานได้ ทำงานร่วมกันแบบ real-time
dbdiagram.io / Supabase Schema	ออกแบบโครงสร้างฐานข้อมูล (ER) ก่อนเขียน SQL 001_init.sql

ตัวอย่างในโครงการ: ออกแบบ schema ตาราง products, categories, orders, cart, users ใน Supabase และวาด flow การ checkout ก่อนลงมือ code

3) Development — การพัฒนา

เครื่องมือ	เหตุผลที่เลือกใช้
VS Code	IDE หลัก มี extension ครบ (ESLint, Tailwind, Git)
Next.js + React + TypeScript	Frontend — App Router, Type safety, SEO, ecosystem ใหญ่
Fastify + TypeScript	Backend API — เร็ว น้ำหนักเบา รองรับ plugin (CORS, Helmet, Rate Limit)

เครื่องมือ	เหตุผลที่เลือกใช้
Tailwind CSS	ทำ UI responsive ได้เร็ว ไม่ต้องเขียน CSS แยกไฟล์
Supabase (PostgreSQL)	ฐานข้อมูล + Auth สำเร็จรูป ลดงาน backend
Git + GitHub	เวอร์ชันคอนโทรล ทำงานเป็นทีม (branch, PR)
Postman / Thunder Client	ทดสอบเรียก API ระหว่างพัฒนา ก่อนต่อกับ Frontend

ตัวอย่างในโครงการ: พัฒนา REST API `/api/v1/products`, `/api/v1/orders`, `/api/v1/payments` แล้วให้ Frontend เรียกผ่าน `NEXT_PUBLIC_API_URL`

4) Testing — การทดสอบระบบ

เครื่องมือ	เหตุผลที่เลือกใช้
Postman / Thunder Client	ทดสอบ API แต่ละ endpoint (status, response, error case)
Jest / Vitest	Unit test ฟังก์ชันสำคัญ เช่น คำนวณราคา/ส่วนลด
Playwright / Cypress	ทดสอบ flow ฝั่ง UI แบบ End-to-End (เช่น สั่งซื้อจนจบ)
Browser DevTools	ตรวจ error, network, console ฝั่ง Frontend
Apache JMeter	ทดสอบโหลด (Load test) จำลองผู้ใช้จำนวนมาก

ตัวอย่างในโครงการ: ทดสอบ flow เพิ่มสินค้าลงตะกร้า → checkout → ชำระผ่าน PromptPay QR → ตรวจสอบสถานะ ให้ครบทุกกรณี

5) Deployment — การนำระบบขึ้นใช้งาน

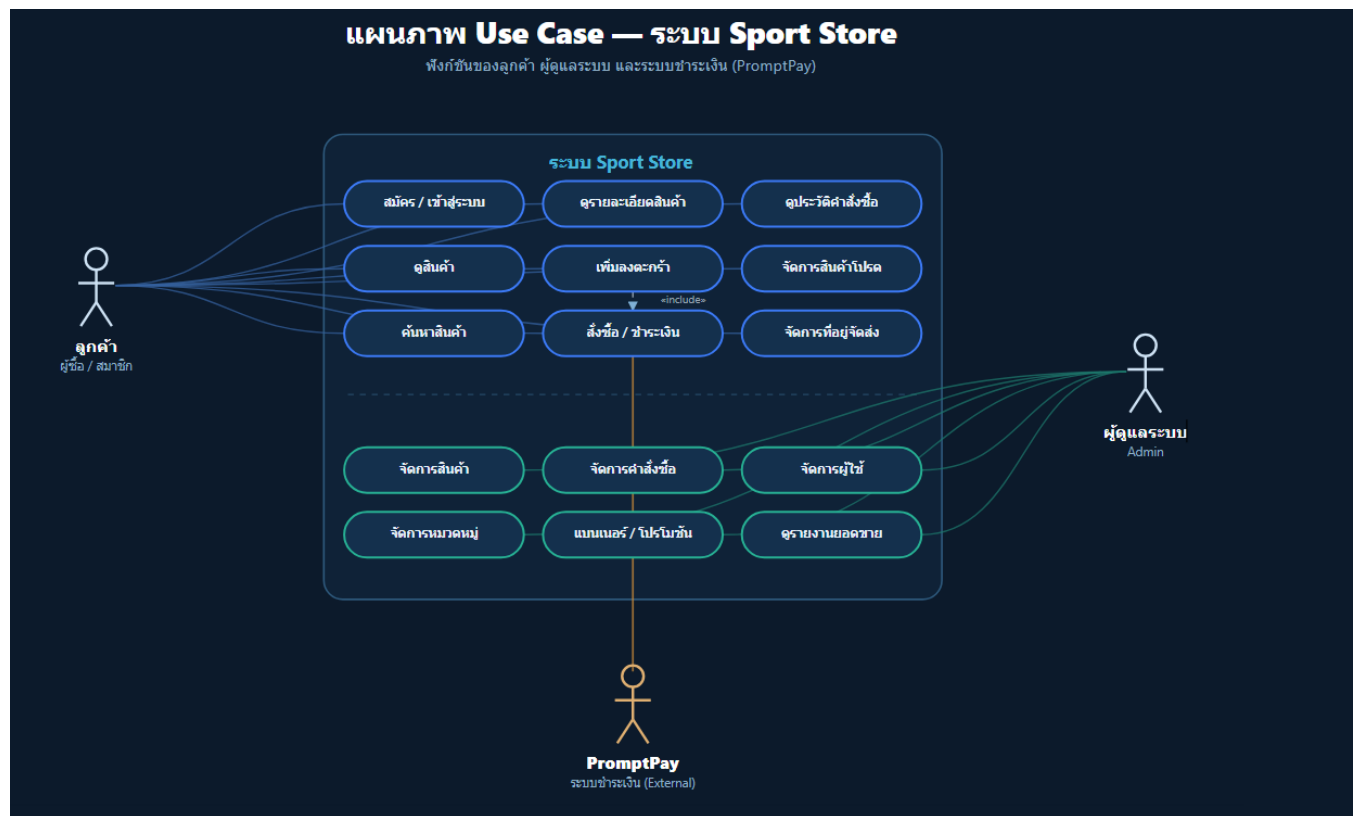
เครื่องมือ	เหตุผลที่เลือกใช้
Docker	สร้าง container ทั้ง Frontend และ Backend ให้รันเหมือนกันทุกเครื่อง
Docker Compose	รันหลาย service พร้อมกัน (frontend + backend + nginx) ด้วยไฟล์เดียว จัดการง่าย
Nginx	ทำ Reverse Proxy — route / ไปที่ Frontend, /api ไปที่ Backend, เสิร์ฟ static, จัดการ SSL/HTTPS และช่วยกระจายโหลด
GitHub Actions	CI/CD — build Docker image และ deploy อัตโนมัติเมื่อ push
Supabase Cloud	โฮสต์ฐานข้อมูล PostgreSQL แบบ managed

ตัวอย่างในโครงการ: เขียน Dockerfile แยกให้ Frontend (Next.js) และ Backend (Fastify), ใช้ docker-compose.yml ผูกทั้งสอง service เข้ากับ Nginx ที่ทำหน้าที่เป็น reverse proxy หน้าที่ (รับ request จากผู้ใช้แล้วส่งต่อไปยัง container ที่ถูกต้อง) ส่วนฐานข้อมูลใช้ Supabase

 **ตำแหน่งแทรกรูปภาพ:** แผนภาพ Deployment — ผู้ใช้ → Nginx → (Frontend container / Backend container) → Supabase

คำถาม 2 — การวิเคราะห์และออกแบบระบบ (5 คะแนน)

2.1 Use Case Diagram



Actors

- ผู้ใช้ทั่วไป (Guest) — ดูสินค้า, ค้นหา, ดูรายละเอียด, จัดการตะกร้า, สมัครสมาชิก
- สมาชิก (Member) — ทุกอย่างของ Guest + เข้าสู่ระบบ, จัดการสินค้าโปรด, จัดการที่อยู่, สั่งซื้อ, ชำระเงิน, ดูประวัติคำสั่งซื้อ
- ผู้ดูแลระบบ (Admin) — จัดการสินค้า/หมวดหมู่/คำสั่งซื้อ/แบนเนอร์/โปรโมชั่น
- ระบบชำระเงิน PromptPay (External) — สร้าง QR / ยืนยันการชำระเงิน

Use Cases หลัก: ดูสินค้า · ค้นหาสินค้า · ดูรายละเอียดสินค้า · จัดการตะกร้า · สมัครสมาชิก · เข้าสู่ระบบ · เพิ่ม/ลบสินค้าโปรด · จัดการที่อยู่จัดส่ง · ทำรายการสั่งซื้อ (Checkout) · ชำระเงินผ่าน PromptPay · ดูประวัติคำสั่งซื้อ

ความสัมพันธ์: Checkout <<include>> เลือกที่อยู่จัดส่ง · Checkout <<include>> ชำระเงิน · ชำระเงิน <<include>> สร้างคำสั่งซื้อ

หน้าที่/ความสำคัญ: แสดงภาพรวมว่าใครทำอะไรได้บ้างในระบบ ใช้กำหนดขอบเขต (scope) และเป็นฐานในการแตก requirement เป็นฟีเจอร์

2.2 Class Diagram



คลาสหลักพร้อม Attributes และ Methods (Operations) — methods อ้างอิงจากฟังก์ชันจริงในระบบ (API + Context ผัง Frontend)

Class	Attributes	Methods
User (ผู้ใช้)	id, email, full_name, phone, avatar_url	register(), login(), logout(), getProfile(), updateProfile(), refreshToken()

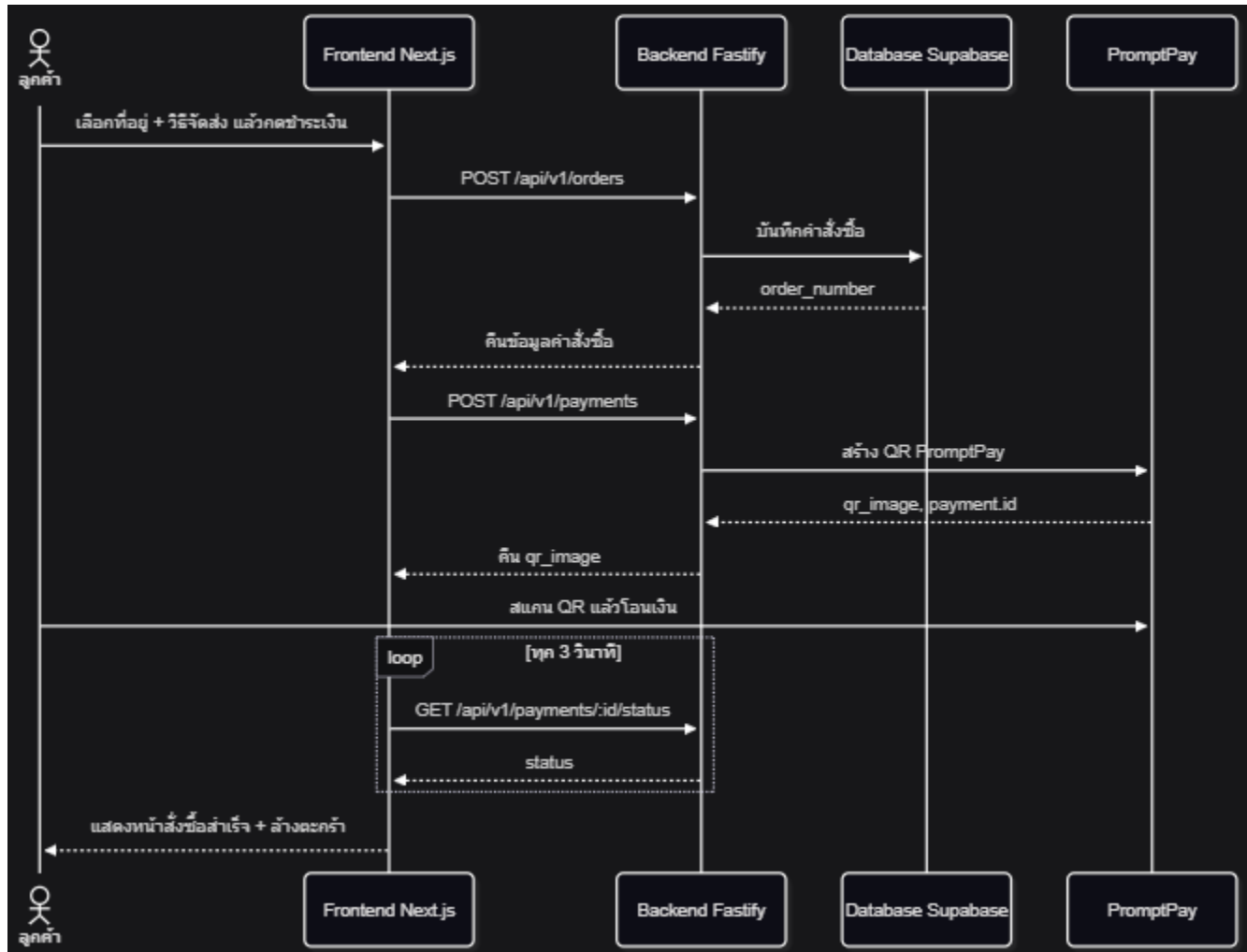
Class	Attributes	Methods
Address (ที่อยู่จัดส่ง)	id, user_id, label, full_name, phone, postal_code, province, amphoe, district, address, is_default	getAll(), create(), update(), delete(), setDefault()
Product (สินค้า)	id, name, slug, price, original_price, brand, images[], tags[], rating, review_count, is_new	getById(), search(), getByCategory(), getPopular()
Category (หมวดหมู่)	id, name, slug, route_path, parent_id, level, sort_order	getTree(), getByRoute(), getChildren()
Cart (ตะกร้าสินค้า)	items[], totalItems, totalPrice	addItem(), removeItem(), updateQuantity(), clearCart(), getTotalPrice()
CartItem (รายการในตะกร้า)	id, product_id, name, brand, image, price, size, quantity	—
Favorite (สินค้าโปรด)	items[], totalItems	addFavorite(), removeFavorite(), toggleFavorite(), isFavorite(), clearFavorites()
Order (คำสั่งซื้อ)	id, order_number, status, shipping_name, shipping_phone, shipping_address, payment_method, subtotal, discount_total, shipping_cost, total, created_at	create(), getHistory(), getDetail(), calculateTotal(), updateStatus()
OrderItem (รายการสินค้าในคำสั่งซื้อ)	id, order_id, product_id, name, brand, size, price, original_price, quantity	getSubtotal()
Payment (การชำระเงิน)	id, order_id, method, amount, ref_amount, status, promptpay_number, expires_at, confirmed_at	createPayment(), generateQR(), checkStatus(), confirm()

ความสัมพันธ์ (Relationships)

- User "1" — "*" Address (ผู้ใช้ 1 คน มีได้หลายที่อยู่)
- User "1" — "*" Order (ผู้ใช้ 1 คน มีได้หลายคำสั่งซื้อ)
- User "1" — "*" Favorite
- Cart "1" — "*" CartItem
- Order "1" — "*" OrderItem
- Product "1" — "*" OrderItem (สินค้า 1 ชิ้น อยู่ในได้หลายคำสั่งซื้อ)
- Order "1" — "1" Payment (คำสั่งซื้อ 1 รายการ มีการชำระเงิน 1 รายการ)
- Category "1" — "*" Category (หมวดหมู่แม่/ลูก — self reference ผ่าน parent_id)
- Product "*" — "1" Category

หน้าที่/ความสำคัญ: แสดงโครงสร้างข้อมูล (attributes), พฤติกรรม/การทำงาน (methods) และความสัมพันธ์ระหว่างคลาส เป็นพิมพ์เขียวของทั้งฐานข้อมูล (mapping ตรงกับตารางใน Supabase) และโครงสร้าง object/ฟังก์ชันในโค้ด

2.3 Sequence Diagram



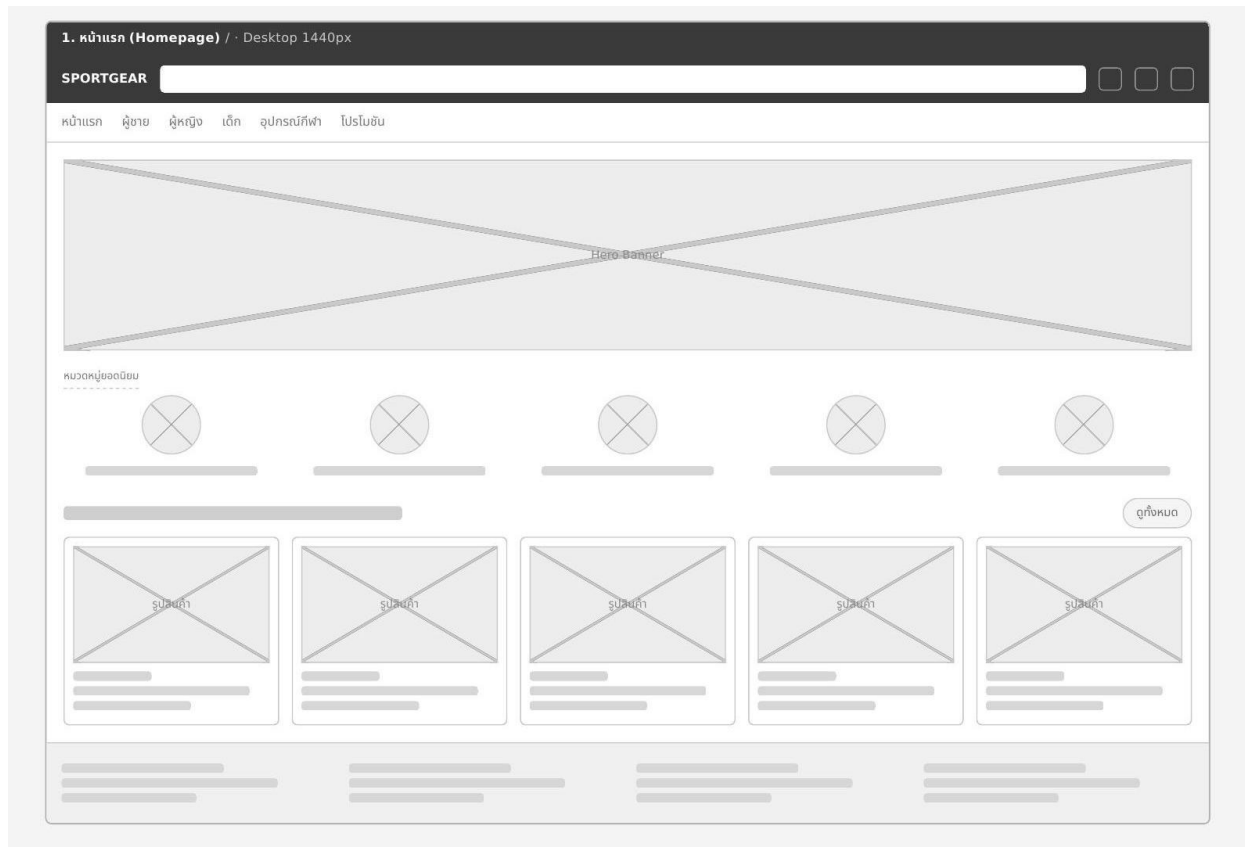
Objects: ลูกค้า → Frontend (Next.js) → Backend API (Fastify) → Database (Supabase) → PromptPay

ลำดับเหตุการณ์ (จาก flow จริงในหน้า checkout)

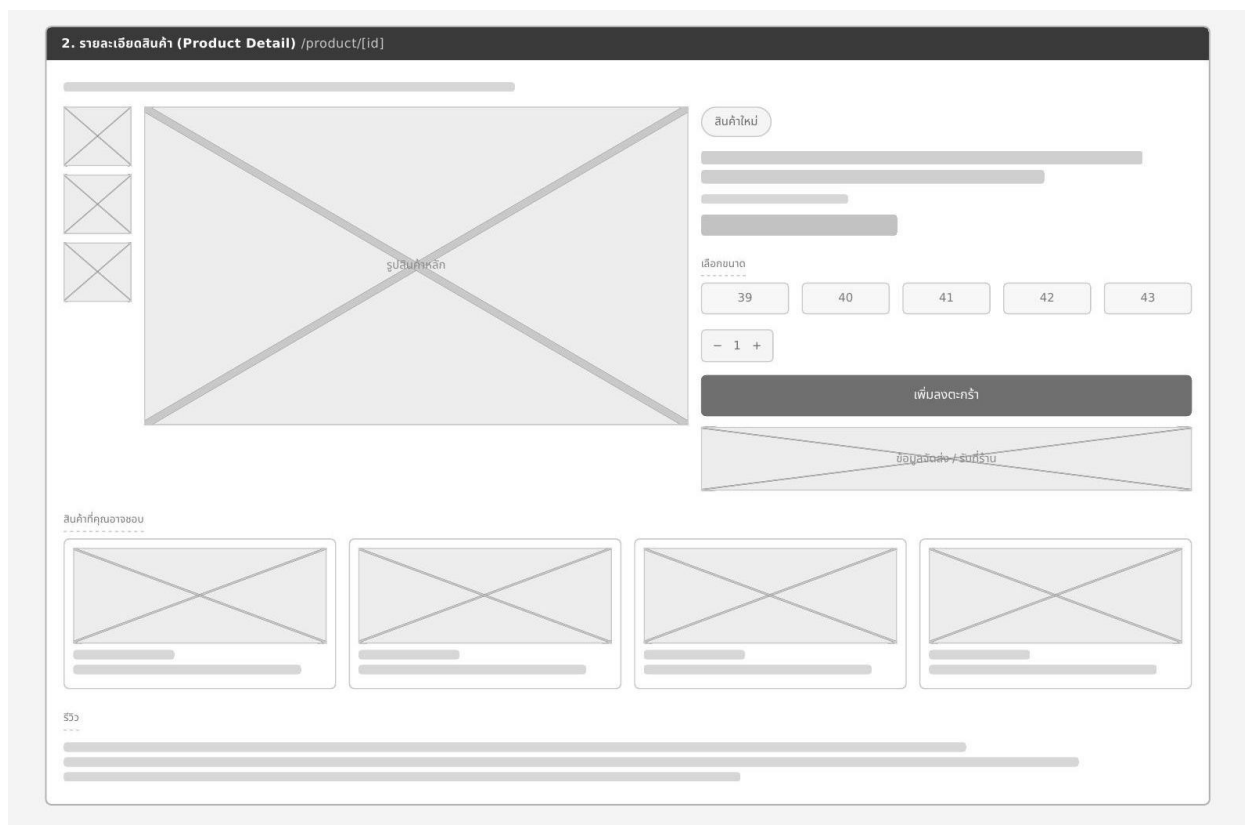
1. ลูกค้าเลือกที่อยู่ + วิธีจัดส่ง → กด "ชำระเงิน"
2. Frontend → POST /api/v1/orders (สร้างคำสั่งซื้อ) → Backend บันทึกลง Database → คืน order_number
3. Frontend → POST /api/v1/payments (สร้างการชำระเงิน) → Backend สร้าง QR PromptPay → คืน qr_image + payment.id
4. ลูกค้าสแกน QR แล้วโอนเงินผ่านแอปธนาคาร
5. Frontend วนเช็คสถานะ GET /api/v1/payments/:id/status ทุก 3 วินาที
6. เมื่อสถานะ = confirmed → แสดงหน้า "สั่งซื้อสำเร็จ" + ล้างตะกร้า

หน้าที่/ความสำคัญ: แสดงลำดับการทำงานตามเวลาระหว่างส่วนต่างๆ ช่วยให้เห็นการรับส่งข้อมูลและ API ที่เกี่ยวข้อง ใช้ตรวจสอบความถูกต้องของ flow ก่อนพัฒนา

2.4 Wireframe



1. หน้าแรก (Homepage)



2. รายละเอียดสินค้า (Product Detail)

3. ตะกร้าสินค้า (Cart) /cart

รูป

-

1

+

รูป

-

1

+

ช่องใส่โค้ดส่วนลด

ดำเนินการชำระเงิน

ข้อบ่งชี้

3. ตะกร้าสินค้า (Cart)

4. ชำระเงิน (Checkout) /checkout · เลือกที่อยู่ → จัดส่ง → ชำระเงิน

1. ที่อยู่

2. จัดส่ง

3. ชำระเงิน

เลือกที่อยู่จัดส่ง

วิธีชำระเงิน

QR

บัตร

โอน

ชำระเงิน

4. ชำระเงิน (Checkout)

5. บัญชีของฉัน (Account) /account

การ์ดแบบ: คำสั่งของฉัน · สบุดที่อยู่ · สินค้าโปรด | ปุ่มออกจากระบบ

— ถึง 5 เฟรมเป็นแบบร่าง (ยังไม่ใส่สี/ภาพจริง) เพื่อยืนยัน layout ก่อนพัฒนา UI จริง —

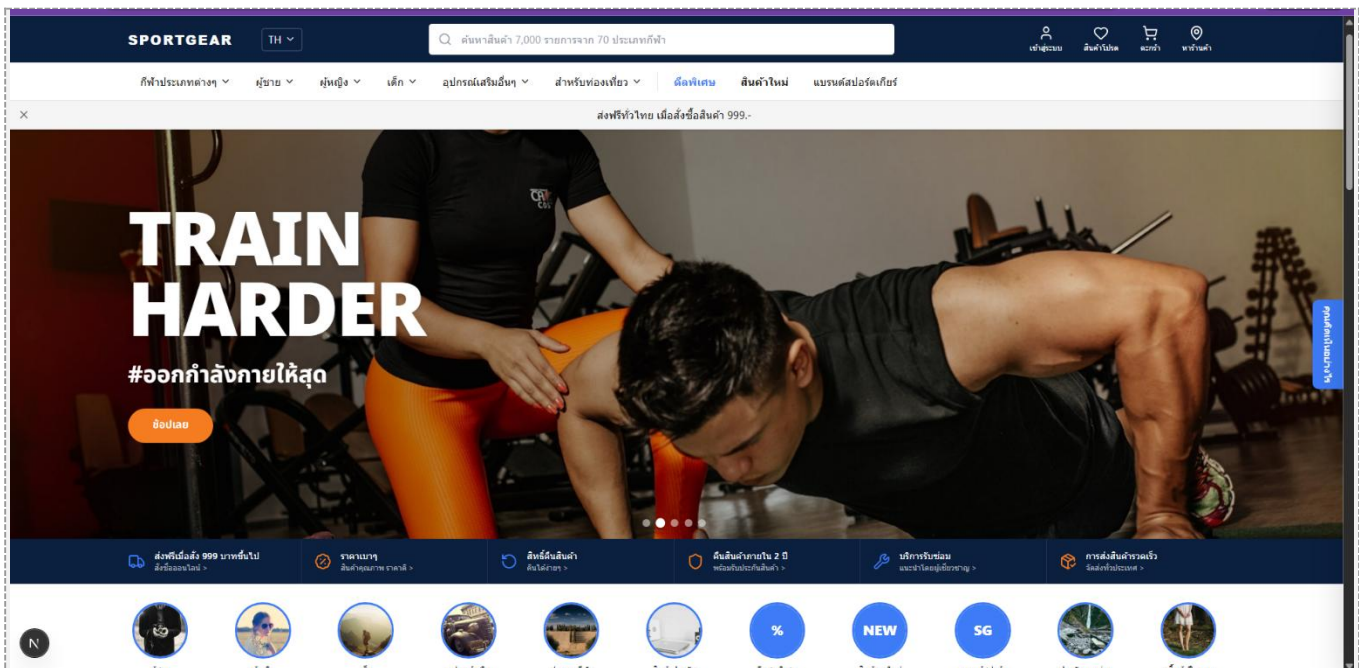
5. บัญชีของฉัน (Account)

หน้าที่ควรมี (โครงสร้าง)

- Homepage: Navbar (โลโก้/ค้นหา/ตะกร้า/บัญชี) → Hero banner → หมวดหมู่ → ดิสคัลดิสคั → กริดสินค้า → Footer
- Product Detail: รูปสินค้า | ชื่อ/ราคา/ไซส์/สี/ปุ่มเพิ่มตะกร้า → รีวิว → สินค้าแนะนำ
- Cart / Checkout: รายการสินค้า | สรุปยอด → เลือกที่อยู่ → วิธีจัดส่ง → ชำระเงิน
- Account: การ์ดโปรไฟล์ + เมนู (คำสั่งซื้อ/ที่อยู่/สินค้าโปรด)

หน้าที่/ความสำคัญ: ร่างโครงหน้าจอบาง (ยังไม่ใส่สี/รายละเอียด) เพื่อตกลง layout และตำแหน่งองค์ประกอบก่อนออกแบบ UI จริง ประหยัดเวลาแก้ไขทีหลัง

2.5 Prototype



Flow ที่ควรทำให้เกิดได้: เข้าหน้าแรก → เลือกสินค้า → เพิ่มลงตะกร้า → checkout → ชำระเงิน → สั่งซื้อสำเร็จ

หน้าที่/ความสำคัญ: ต้นแบบเหมือนจริง (สี/ฟอนต์/ปุ่มกดได้) ใช้ทดสอบประสบการณ์ผู้ใช้ (UX) และนำเสนอลูกค้า/อาจารย์ก่อนพัฒนาเต็มรูปแบบ

คำถาม 3 — การวิเคราะห์ปัญหาและการทดสอบระบบ (1 คะแนน)

สถานการณ์: ช่วง Flash Sale ผู้ใช้เยอะ → API Response ช้า → ระบบใช้งานไม่ได้

เครื่องมือที่เลือกใช้

ประเภท	เครื่องมือ	ใช้ทำอะไร
Load Testing	Apache JMeter	จำลองผู้ใช้จำนวนมากเพื่อสร้างปัญหาช้า และวัดจุดที่ระบบรับไม่ไหว
ทดสอบ/ตรวจ Endpoint	Postman	ยิงเรียกแต่ละ endpoint เพื่อดูเวลาตอบกลับ (Response Time) และ payload หา endpoint ที่ช้าแบบเจาะจง
APM / Monitoring	Sentry, Grafana + Prometheus	ดู latency (p95/p99), error rate, endpoint ที่ช้า
Backend Logging	Pino (Fastify logger)	ดู request ที่ใช้เวลานาน หา endpoint ต้นเหตุ
Database	Supabase Query Insights, EXPLAIN ANALYZE, pg_stat_statements	หา query ที่ช้า / ขาด index / N+1
Frontend	Browser DevTools (Network tab)	ยืนยันว่าช้าที่ API จริง ไม่ใช่ฝั่ง client

ขั้นตอนการดำเนินงาน

- จำลองปัญหา — ใช้ Apache JMeter ยิงโหลดไปที่ endpoint ที่คนใช้เยอะช่วง Flash Sale (เช่น /api/v1/products, /api/v1/orders) เพื่อ reproduce ปัญหา
- วัดและระบุจุดช้า — ดูจาก APM/log ว่า endpoint ไหน latency สูง, ค่า p95/p99 เท่าไร
- วิเคราะห์สาเหตุ — แยกว่าปัญหายู่ที่ Database (query ช้า/ไม่มี index/ดึงข้อมูลเกินจำเป็น/N+1 query), Backend (connection pool เต็ม, ไม่มี cache, CPU ต้น) หรือ Infrastructure (rate limit, scale ไม่พอ)
- ยืนยันต้นเหตุ (root cause) — เช่น query สินค้าไม่มี index + ไม่มี cache + Frontend ยิง request ซ้ำถี่เกินไป

แนวทางแก้ไขเบื้องต้น

- เพิ่ม Index ในคอลัมน์ที่ query บ่อย (เช่น category_id, slug) และ optimize query + ใส่ pagination
- ทำ Caching — Redis/Upstash หรือ HTTP cache header สำหรับข้อมูลที่ไม่เปลี่ยนบ่อย (สินค้า/หมวดหมู่)
- Connection Pooling — ใช้ Supabase Pooler (PgBouncer) กันคอนเนกชันเต็ม
- Rate Limiting — จำกัด request ต่อ IP (มี Rate Limit plugin ใน Fastify อยู่แล้ว)
- Scaling — เพิ่มจำนวน container backend หลัง Nginx (load balancing) เพื่อรับโหลดช่วงพีค
- ลดขนาด payload — ส่งเฉพาะฟิลด์ที่จำเป็น

